FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO – FECAP

Curso de Análise e Desenvolvimento de Sistemas – 2º semestre de 2026

Projeto Interdisciplinar

KFKA / AlphaTech

Views no banco de dados da Plataforma de Acompanhamento Escolar

Disciplina: Banco de Dados

Atividade: Entrega 2 – Documento de criação de Views

Equipe AlphaTech

Daniel Câmara Freitas

Enzo Lemos de Souza do Nascimento

Igor Araujo

Matheus Teixeira da Rocha Lima

Empresa parceira: KFKA

Orientadores: Adriano Felix Valente, Chico Escobar, Eduardo Savino Gomes, José Carlos Buesso e Ronaldo Araujo Pinto

São Paulo

Novembro de 2026

1. Introdução

A KFKA, empresa de desenvolvimento de software, propôs às equipes do 2º semestre de ADS a criação de uma aplicação web para escolas de Ensino Fundamental. A equipe AlphaTech desenvolveu uma plataforma em que o professor registra o acompanhamento bimestral de cada aluno (média, faltas, parecer descritivo e tags), a coordenação revisa e publica esse registro, e o responsável consulta somente o que foi publicado.

Os dados ficam em um banco MySQL 8 definido pelo script da equipe `database/kfka_banco.sql`. O modelo é normalizado: para exibir um único acompanhamento com nomes legíveis é preciso reunir dez tabelas (o próprio acompanhamento, aluno, turma, disciplina, área, professor, usuário, bimestre, tag e a tabela de ligação acompanhamento_tag). Para que a aplicação não precise repetir essas junções em cada consulta, o script cria seis *views* na seção 5 (“VIEWS”).

Uma view é uma consulta SELECT armazenada no banco com um nome. Ela se comporta como uma tabela virtual: pode ser usada em um SELECT, em JOINs e com WHERE, mas não guarda dados próprios. A cada consulta, o MySQL executa a consulta da view sobre os dados atuais das tabelas, de modo que o resultado está sempre atualizado.

2. Objetivo

Este documento apresenta as views criadas no banco da plataforma, o código SQL de cada uma, exemplos de consulta com resultados reais e os pontos do sistema que as utilizam. As views foram implementadas com quatro objetivos:

- **Organizar consultas complexas.** A junção das dez tabelas fica escrita uma única vez em `vw_acompanhamento_completo`, e as demais consultas partem dela.
- **Concentrar regras de negócio no banco.** A regra “o responsável só vê registros PUBLICADOS de alunos aos quais tem vínculo autorizado” está dentro de `vw_relatorio_responsavel`. Toda tela que lê essa view já recebe os dados filtrados, o que protege dados de crianças e adolescentes (LGPD).
- **Reutilizar consultas.** A mesma view atende a várias rotas da API (boletim, PDF, painel, pendências, ciência), sem duplicar SQL.
- **Simplificar indicadores.** As views de indicadores entregam médias, contagens e frequência de tags já agregadas por turma, disciplina e bimestre, prontas para o painel.

3. Tabelas envolvidas

As views usam as tabelas abaixo, todas definidas em database/kfka_banco.sql. As colunas listadas são as que aparecem nas views.

Tabela	O que guarda (colunas usadas)	Views que a usam
acompanhamento	Registro bimestral do aluno: aluno_id, turma_id, disciplina_id, professor_id, bimestre_id, media, faltas, descricao, status, versao, revisor_id, enviado_em, publicado_em	vw_acompanhamento_completo, vw_frequencia_tags
bimestre	ano_letivo e numero (1 a 4) do bimestre	vw_acompanhamento_completo, vw_frequencia_tags
turma	nome e serie da turma	vw_acompanhamento_completo, vw_frequencia_tags
aluno	nome e matricula	vw_acompanhamento_completo
disciplina	nome e area_id	vw_acompanhamento_completo
area	nome da área do conhecimento	vw_acompanhamento_completo
professor	usuario_id do professor	vw_acompanhamento_completo
usuario	nome do professor, do revisor e do responsável	vw_acompanhamento_completo, vw_retornos_responsaveis
tag / acompanhamento_tag	Tags de desempenho e a ligação N:N com o acompanhamento	vw_acompanhamento_completo, vw_frequencia_tags
aluno_responsavel	Vínculo aluno × responsável, com a coluna autorizado	vw_relatorio_responsavel
ciencia_responsavel	Confirmação de ciência: data_ciencia e observacao	vw_relatorio_responsavel, vw_retornos_responsaveis
responsavel	usuario_id do responsável	vw_retornos_responsaveis

Tabela 1 – Tabelas que participam das views.

4. Views implementadas

O banco possui seis views. Quatro delas são consultadas diretamente pela API; duas ficam disponíveis para consultas diretas no banco (ver seção 7.3).

View	Finalidade	Construída sobre	Uso na API
`vw_acompanhamento_completo`	Visão completa de cada acompanhamento, com nomes legíveis e tags	10 tabelas	Sim (base de tudo)
`vw_relatorio_responsavel`	Boletim do responsável: só PUBLICADOS e só alunos vinculados	vw_acompanhamento_completo + 2 tabelas	Sim
`vw_fila_revisao`	Registros aguardando revisão e há quantas horas	vw_acompanhamento_completo	Não (consulta direta)

View	Finalidade	Construída sobre	Uso na API
`vw_indicadores_turma_disciplina`	Indicadores por turma × disciplina × bimestre	vw_acompanhamento_completo	Sim
`vw_frequencia_tags`	Quantas vezes cada tag foi usada por turma e bimestre	5 tabelas	Sim
`vw_retornos_responsaveis`	Ciências e observações enviadas pelas famílias	vw_acompanhamento_completo + 3 tabelas	Não (consulta direta)

Tabela 2 – Views do banco de dados (database/kfka_banco.sql, seção 5).

4.1 vw_acompanhamento_completo

É a view principal. Para cada linha da tabela acompanhamento, ela traz os nomes do aluno, da turma, da disciplina, da área, do professor e do revisor, o ano letivo e o número do bimestre, além das tags do registro reunidas em um único texto. Todas as outras views, exceto vw_frequencia_tags, são construídas sobre ela.

Relacionamentos

- `acompanhamento.bimestre_id = bimestre.id` (INNER JOIN)
- `acompanhamento.turma_id = turma.id`, `acompanhamento.aluno_id = aluno.id`, `acompanhamento.disciplina_id = disciplina.id` (INNER JOIN)
- `disciplina.area_id = area.id` (INNER JOIN)
- `acompanhamento.professor_id = professor.id` e `professor.usuario_id = usuario.id` (nome do professor)
- `acompanhamento.revisor_id = usuario.id` com LEFT JOIN, porque o revisor só existe depois da revisão
- `acompanhamento_tag` + `tag` em uma subconsulta correlacionada que concatena as tags

4.2 vw_relatorio_responsavel

Entrega ao responsável o boletim do filho. A view parte de vw_acompanhamento_completo e aplica duas regras do enunciado da KFKA: somente registros com status PUBLICADO (“somente registros publicados poderão ser visualizados pelo Pai/Responsável”) e somente alunos com vínculo autorizado (“um aluno poderá ter mais de um responsável autorizado”). Também mostra se aquele responsável já confirmou a ciência e a observação que enviou.

Relacionamentos e filtros

- `vw_acompanhamento_completo.aluno_id = aluno_responsavel.aluno_id` com `aluno_responsavel.autorizado = TRUE` (INNER JOIN)
- `ciencia_responsavel` por acompanhamento e responsável (LEFT JOIN: a ciência pode ainda não existir)
- `WHERE v.status = 'PUBLICADO'`

4.3 vw_fila_revisao

Lista os registros com status ENVIADO ou EM_REVISAO, isto é, os que aguardam a coordenação, e calcula há quantas horas cada um espera (TIMESTAMPDIFF entre enviado_em e o momento atual). Usa apenas vw_acompanhamento_completo, com o filtro WHERE status IN ('ENVIADO','EM_REVISAO').

4.4 vw_indicadores_turma_disciplina

Agrupa os registros por ano letivo, bimestre, turma, disciplina e professor e calcula: quantidade de registros, quantidade publicada, média geral (arredondada em duas casas), menor e maior média, quantos alunos estão abaixo de 6 e o total de faltas. Registros CANCELADOS ficam de fora (WHERE status <> 'CANCELADO').

4.5 vw_frequencia_tags

Conta quantas vezes cada tag foi atribuída, por ano letivo, bimestre e turma. Parte da tabela de ligação acompanhamento_tag e faz INNER JOIN com tag, acompanhamento, bimestre e turma, desconsiderando registros CANCELADOS.

4.6 vw_retornos_responsaveis

Reúne as confirmações de ciência e as observações enviadas pelas famílias, com o professor, a turma, o aluno, a disciplina e o bimestre do registro. Faz INNER JOIN de ciencia_responsavel com responsavel, usuario (nome do responsável) e vw_acompanhamento_completo.

5. Implementação SQL

Os trechos abaixo são o código real de criação das views, copiado do arquivo database/kfka_banco.sql sem alterações. Os comentários “-- [SITE]” fazem parte do script: marcam as colunas de faltas que a equipe acrescentou ao modelo original.

5.1 vw_acompanhamento_completo

```
CREATE VIEW vw_acompanhamento_completo AS
SELECT
    a.id                        AS acompanhamento_id,
    b.ano_letivo,
    b.numero                    AS bimestre,
    t.id                        AS turma_id,
    t.nome                      AS turma,
    t.serie,
    al.id                       AS aluno_id,
    al.matricula,
    al.nome                     AS aluno,
    ar.nome                     AS area,
    d.id                        AS disciplina_id,
    d.nome                      AS disciplina,
    p.id                        AS professor_id,
    up.nome                     AS professor,
    a.media,
    a.faltas,                                           -- [SITE]
    a.descricao,
    a.status,
    a.versao,
    (SELECT GROUP_CONCAT(tg.nome ORDER BY tg.nome SEPARATOR ', ')
       FROM acompanhamento_tag at2
       JOIN tag tg ON tg.id = at2.tag_id
      WHERE at2.acompanhamento_id = a.id) AS tags,
    ur.nome                     AS revisor,
    a.enviado_em,
    a.publicado_em
FROM acompanhamento a
JOIN bimestre   b  ON b.id  = a.bimestre_id
JOIN turma      t  ON t.id  = a.turma_id
JOIN aluno      al ON al.id = a.aluno_id
JOIN disciplina d  ON d.id  = a.disciplina_id
JOIN area       ar ON ar.id = d.area_id
JOIN professor  p  ON p.id  = a.professor_id
JOIN usuario    up ON up.id = p.usuario_id
LEFT JOIN usuario ur ON ur.id = a.revisor_id;
```

Fonte: database/kfka_banco.sql, linhas 418 a 454 (código copiado sem alterações).

Principais comandos

- **CREATE VIEW … AS SELECT**: grava a consulta no banco com o nome da view.
- **Aliases (AS)**: renomeiam colunas repetidas, como `t.nome AS turma` e `al.nome AS aluno`, para que a view tenha nomes claros.
- **INNER JOIN**: liga o acompanhamento às tabelas obrigatórias. Um registro só aparece se todas as chaves estrangeiras existirem.
- **LEFT JOIN usuario ur**: mantém o registro mesmo sem revisor (rascunhos e registros enviados).
- **Subconsulta com GROUP_CONCAT**: para cada acompanhamento, junta os nomes das tags em ordem alfabética, separados por vírgula, em uma única coluna `tags`. Assim a relação N:N não multiplica as linhas da view.

5.2 vw_relatorio_responsavel

```
CREATE VIEW vw_relatorio_responsavel AS
SELECT
    rel.responsavel_id,
    v.acompanhamento_id,
    v.ano_letivo, v.bimestre, v.turma, v.aluno_id, v.aluno,
    v.area, v.disciplina, v.professor, v.media, v.faltas, v.descricao, v.tags,   -- [SITE] faltas
    v.publicado_em,
    c.data_ciencia,
    c.observacao AS retorno_responsavel
FROM vw_acompanhamento_completo v
JOIN aluno_responsavel rel ON rel.aluno_id = v.aluno_id AND rel.autorizado = TRUE
LEFT JOIN ciencia_responsavel c
       ON c.acompanhamento_id = v.acompanhamento_id
      AND c.responsavel_id    = rel.responsavel_id
WHERE v.status = 'PUBLICADO';
```

Fonte: database/kfka_banco.sql, linhas 460 a 474 (código copiado sem alterações).

Principais comandos

- **View sobre view**: o FROM usa `vw_acompanhamento_completo`, reaproveitando as oito junções.
- **JOIN com condição composta**: `rel.autorizado = TRUE` faz parte do JOIN; responsável sem autorização não gera linha.
- **LEFT JOIN ciencia_responsavel**: a data da ciência fica NULL enquanto o responsável não confirma.
- **WHERE v.status = 'PUBLICADO'**: rascunhos, registros em revisão e devolvidos nunca chegam à família.

5.3 vw_fila_revisao

```
CREATE VIEW vw_fila_revisao AS
SELECT acompanhamento_id, ano_letivo, bimestre, turma, aluno, disciplina,
       professor, media, faltas, status, enviado_em,          -- [SITE] faltas
       TIMESTAMPDIFF(HOUR, enviado_em, NOW()) AS horas_aguardando
FROM vw_acompanhamento_completo
WHERE status IN ('ENVIADO','EM_REVISAO');
```

Fonte: database/kfka_banco.sql, linhas 477 a 482 (código copiado sem alterações).

- **TIMESTAMPDIFF(HOUR, enviado_em, NOW())**: diferença em horas entre o envio e o momento da consulta.
- **WHERE status IN ('ENVIADO','EM_REVISAO')**: só o que depende da coordenação.

5.4 vw_indicadores_turma_disciplina

```
CREATE VIEW vw_indicadores_turma_disciplina AS
SELECT
    ano_letivo, bimestre, turma, disciplina, professor,
    COUNT(*)                          AS qtd_registros,
    SUM(status = 'PUBLICADO')         AS qtd_publicados,
    ROUND(AVG(media), 2)              AS media_geral,
    MIN(media)                        AS menor_media,
    MAX(media)                        AS maior_media,
    SUM(media < 6)                    AS qtd_abaixo_de_6,
    SUM(faltas)                       AS total_faltas          -- [SITE]
FROM vw_acompanhamento_completo
WHERE status <> 'CANCELADO'
GROUP BY ano_letivo, bimestre, turma, disciplina, professor;
```

Fonte: database/kfka_banco.sql, linhas 485 a 497 (código copiado sem alterações).

- **GROUP BY** com **COUNT, AVG, MIN, MAX e SUM**: uma linha por turma × disciplina × bimestre.
- **SUM(status = 'PUBLICADO') e SUM(media < 6)**: no MySQL, uma comparação vale 1 ou 0; somá-la conta quantas linhas atendem à condição.
- **ROUND(AVG(media), 2)**: média geral com duas casas decimais.

5.5 vw_frequencia_tags

```
CREATE VIEW vw_frequencia_tags AS
SELECT
    b.ano_letivo, b.numero AS bimestre, t.nome AS turma, tg.nome AS tag,
    COUNT(*) AS ocorrencias
FROM acompanhamento_tag at2
JOIN tag            tg ON tg.id = at2.tag_id
JOIN acompanhamento a  ON a.id  = at2.acompanhamento_id
JOIN bimestre       b  ON b.id  = a.bimestre_id
JOIN turma          t  ON t.id  = a.turma_id
WHERE a.status <> 'CANCELADO'
GROUP BY b.ano_letivo, b.numero, t.nome, tg.nome;
```

Fonte: database/kfka_banco.sql, linhas 500 a 510 (código copiado sem alterações).

- **COUNT(*) com GROUP BY**: conta as atribuições de cada tag por ano, bimestre e turma, a partir da tabela de ligação.

5.6 vw_retornos_responsaveis

```
CREATE VIEW vw_retornos_responsaveis AS
SELECT
    v.professor_id, v.professor, v.turma, v.aluno, v.disciplina, v.bimestre,
    u.nome AS responsavel, c.data_ciencia, c.observacao
FROM ciencia_responsavel c
JOIN responsavel r ON r.id = c.responsavel_id
JOIN usuario     u ON u.id = r.usuario_id
JOIN vw_acompanhamento_completo v ON v.acompanhamento_id = c.acompanhamento_id;
```

Fonte: database/kfka_banco.sql, linhas 513 a 520 (código copiado sem alterações).

- **INNER JOIN com a view principal**: cada ciência recebe os dados do registro a que se refere; o nome do responsável vem de `usuario` por meio de `responsavel`.

6. Exemplos de consulta e resultado

Ambiente da execução. Os três scripts do projeto (`kfka_banco.sql`, `kfka_expansao.sql` e `kfka_adequacao.sql`) foram executados, na ordem e sem alterações, no Dolt 2.7.2, um servidor compatível com MySQL 8. Os resultados abaixo foram copiados dessa execução e vêm da carga de demonstração do próprio script (dados fictícios). As colunas de data foram omitidas dos exemplos porque a carga registra o momento da execução (NOW()).

6.1 O efeito da regra de publicação

Primeiro, todos os registros do aluno Lucas Ribeiro (aluno_id = 1) na view principal, que não aplica a regra de publicação:

```
SELECT acompanhamento_id, aluno, bimestre, disciplina, status FROM vw_acompanhamento_completo
WHERE aluno_id = 1 ORDER BY acompanhamento_id
```

acompanhamento_id	aluno	bimestre	disciplina	status
1	Lucas Ribeiro	1	Língua Portuguesa	PUBLICADO
5	Lucas Ribeiro	1	História	PUBLICADO
6	Lucas Ribeiro	4	Língua Portuguesa	RASCUNHO
7	Lucas Ribeiro	1	Matemática	PUBLICADO
12	Lucas Ribeiro	1	Ciências	PUBLICADO

Tabela 3 – Resultado em vw_acompanhamento_completo: 5 registros, um deles em RASCUNHO.

Agora a mesma família pela view do responsável. Ana Paula Ribeiro (responsavel_id = 1) é responsável por Lucas e Beatriz. A consulta tem a mesma forma usada pela rota GET /api/boletim (seção 7.1), com menos colunas:

```
SELECT aluno, bimestre, disciplina, professor, media, faltas, tags FROM vw_relatorio_responsavel
WHERE responsavel_id = 1 AND ano_letivo = 2026 ORDER BY aluno, bimestre, disciplina
```

aluno	bimestre	disciplina	professor	media	faltas	tags
Beatriz Ribeiro	1	Língua Portuguesa	Carlos Menezes	7.00	2	Entrega em dia
Lucas Ribeiro	1	Ciências	Ricardo Tanaka	8.00	1	Participativo, Trabalho em equipe
Lucas Ribeiro	1	História	Carlos Menezes	8.00	1	Participativo, Trabalho em equipe
Lucas Ribeiro	1	Língua Portuguesa	Carlos Menezes	8.50	1	Entrega em dia, Participativo
Lucas Ribeiro	1	Matemática	Fernanda Lopes	9.00	1	Destaque, Entrega em dia

Tabela 4 – Resultado em vw_relatorio_responsavel para responsavel_id = 1.

O resultado mostra a view aplicando a regra: o registro 6 de Lucas (Língua Portuguesa, 4º bimestre, status RASCUNHO) não aparece para a família, e os registros de Beatriz entram porque ela também está vinculada a Ana. O registro de Inglês de Beatriz também não aparece,

porque está DEVOLVIDO. Em nenhum momento a consulta precisou repetir o filtro de status ou de vínculo: ele está na view.

6.2 Indicadores por turma e disciplina

Consulta equivalente à usada pelo painel da coordenação (GET /api/dashboard/resumo):

```
SELECT bimestre, turma, disciplina, qtd_registros, qtd_publicados, media_geral, menor_media, maior_media, qtd_abaixo_de_6, total_faltas FROM vw_indicadores_turma_disciplina WHERE ano_letivo = 2026 ORDER BY bimestre DESC, turma, disciplina
```

bim.	turma	disciplina	qtd_ registros	qtd_ publicados	media_ geral	menor_ media	maior_ media	qtd_abaixo _de_6	total_ faltas
4	6º A	Língua Portuguesa	1	0	8.8	8.80	8.80	0	0
1	4º B	Inglês	1	0	8.5	8.50	8.50	0	0
1	4º B	Língua Portuguesa	1	1	7	7.00	7.00	0	2
1	4º B	Matemática	2	0	6.75	6.00	7.50	0	5
1	6º A	Ciências	1	1	8	8.00	8.00	0	1
1	6º A	História	1	1	8	8.00	8.00	0	1
1	6º A	Língua Portuguesa	3	3	7.83	5.50	9.50	1	5
1	6º A	Matemática	3	3	7.77	4.50	9.80	1	6

Tabela 5 – Resultado em vw_indicadores_turma_disciplina para o ano letivo 2026.

Exemplo de leitura: no 1º bimestre, a turma 6º A tem 3 registros de Matemática, todos publicados, com média geral 7,77, menor média 4,50, maior 9,80 e um aluno abaixo de 6. Na 4º B, Matemática tem 2 registros e nenhum publicado (um enviado e um rascunho), o que mostra à coordenação onde o fluxo está parado.

Observação: o Dolt exibe media_geral sem zeros à direita (por exemplo, 8.8 e 7). No MySQL, ROUND(…, 2) devolve o valor com duas casas decimais (8.80 e 7.00); o valor numérico é o mesmo.

6.3 Tags mais frequentes

Consulta usada pelo painel para mostrar as seis tags mais atribuídas no ano:

```
SELECT tag, SUM(ocorrencias) AS total FROM vw_frequencia_tags WHERE ano_letivo = 2026 GROUP BY tag ORDER BY total DESC, tag LIMIT 6
```

tag	total
Entrega em dia	6
Participativo	4
Destaque	3
Precisa de reforço	3
Trabalho em equipe	2
Dificuldade de atenção	1

Tabela 6 – Resultado da consulta sobre vw_frequencia_tags para 2026.

7. Integração com o sistema

A API da plataforma (Node.js com Express) consulta as views pelo módulo server/db, sempre com parâmetros (?), nunca com valores concatenados no SQL. A tabela abaixo lista os pontos do código-fonte que leem cada view.

View	Arquivo	Rota da API	Funcionalidade (tela)
`vw_acompanhamento_completo`	server/routes/acompanhamentos/revisao.js	GET /api/acompanhamentos/fila	Tela Revisão (coordenação) e situação dos lançamentos do professor
	server/routes/acompanhamentos/historico.js	GET /api/acompanhamentos/:id/historico	Histórico do lançamento
	server/routes/boletim.routes.js	GET /api/boletim/pdf (coordenação)	PDF do boletim gerado pela coordenação
	server/routes/documentos.routes.js	GET /api/documentos/:id/conteudo	Documentos emitidos (boletim e histórico escolar)
	server/pendencias/coordenacao.js e professor.js	GET /api/pendencias	Tela Pendências (revisões aguardando, devolvidos, rascunhos)
`vw_relatorio_responsavel`	server/routes/boletim.routes.js	GET /api/boletim, GET /api/boletim/anos, GET /api/boletim/pdf	Tela Boletim e PDF do responsável
	server/routes/acompanhamentos/historico.js	POST /api/acompanhamentos/:id/ciencia	Confirmação de ciência (só de relatório publicado e vinculado)
	server/routes/dashboard.routes.js	GET /api/dashboard/resumo	Painel do responsável (publicados, aguardando ciência, média, faltas)
	server/routes/perfil.routes.js	GET /api/perfil/meus-dados	“Baixar meus dados” (LGPD)
	server/pendencias/responsavel.js	GET /api/pendencias	Boletins novos e ciências pendentes
`vw_indicadores_turma_disciplina`	server/routes/dashboard.routes.js	GET /api/dashboard/resumo	Painel da coordenação e do professor
`vw_frequencia_tags`	server/routes/dashboard.routes.js	GET /api/dashboard/resumo	Tags mais frequentes no painel

Tabela 7 – Uso das views no código-fonte.

7.1 Exemplo: boletim do responsável

Na rota GET /api/boletim, o identificador do responsável vem do token de login (req.usuario.responsavelId), e não de um parâmetro enviado pelo navegador. Combinado com a view, isso garante que cada família receba apenas os registros publicados dos próprios filhos:

```
const registros = await db.query(
  `SELECT acompanhamento_id AS id, aluno_id, bimestre, area, disciplina, professor, media, faltas,
          descricao, tags, publicado_em, data_ciencia, retorno_responsavel
     FROM vw_relatorio_responsavel
    WHERE responsavel_id = ? AND ano_letivo = ?
    ORDER BY bimestre, disciplina`,
  [rid, ano]
);
```

Fonte: server/routes/boletim.routes.js, linhas 240 a 247.

7.2 Exemplo: indicadores do painel

O painel da equipe lê os indicadores já agregados. Quando o usuário é professor, o filtro por professor limita o resultado às turmas dele:

```
const indicadores = await db.query(
  `SELECT bimestre, turma, disciplina, professor, qtd_registros, qtd_publicados, media_geral,
          menor_media, maior_media, qtd_abaixo_de_6, total_faltas
     FROM vw_indicadores_turma_disciplina
    WHERE ano_letivo = ? AND (? IS NULL OR professor = ?)
    ORDER BY bimestre DESC, turma, disciplina`,
  [ano, nomeProfessor, nomeProfessor]
);
```

Fonte: server/routes/dashboard.routes.js, linhas 87 a 94.

7.3 Views sem consumo direto pela API

As views vw_fila_revisao e vw_retornos_responsaveis estão criadas e funcionando no banco (seção 5), mas a versão atual da API não as consulta diretamente:

- A tela de Revisão precisa de colunas que vw_fila_revisao não traz (parecer, tags, versão e revisor), de outros filtros (devolvidos, publicados, rascunhos) e de paginação. Por isso, `revisao.js` consulta vw_acompanhamento_completo diretamente, com o mesmo critério de status.
- Os retornos das famílias são listados em `server/routes/relatorios.routes.js`, com uma consulta própria que aceita filtros e paginação.

As duas views continuam úteis para consultas diretas da coordenação ou da equipe técnica, por exemplo pelo MySQL Workbench.

8. Conclusão

As views tornaram o banco da plataforma KFKA mais organizado e mais seguro. A junção das dez tabelas do acompanhamento foi escrita uma única vez, em vw_acompanhamento_completo, e é reutilizada por outras quatro views e por várias rotas da

API. Isso reduziu a repetição de SQL e o risco de uma consulta esquecer uma junção ou um filtro.

A principal contribuição está em vw_relatorio_responsavel: a regra de que a família só vê registros publicados de alunos com vínculo autorizado ficou no próprio banco. O exemplo da seção 6.1 mostra o efeito: o rascunho e o registro devolvido não aparecem para a responsável, sem que a consulta precise repetir essa condição.

As views de indicadores e de tags entregam ao painel dados já agregados, prontos para exibição. Como uma view não armazena dados, todos esses resultados refletem o estado atual das tabelas a cada consulta. Assim, a equipe atendeu à exigência de Banco de Dados da Entrega 2 com views efetivamente implementadas e usadas no sistema.